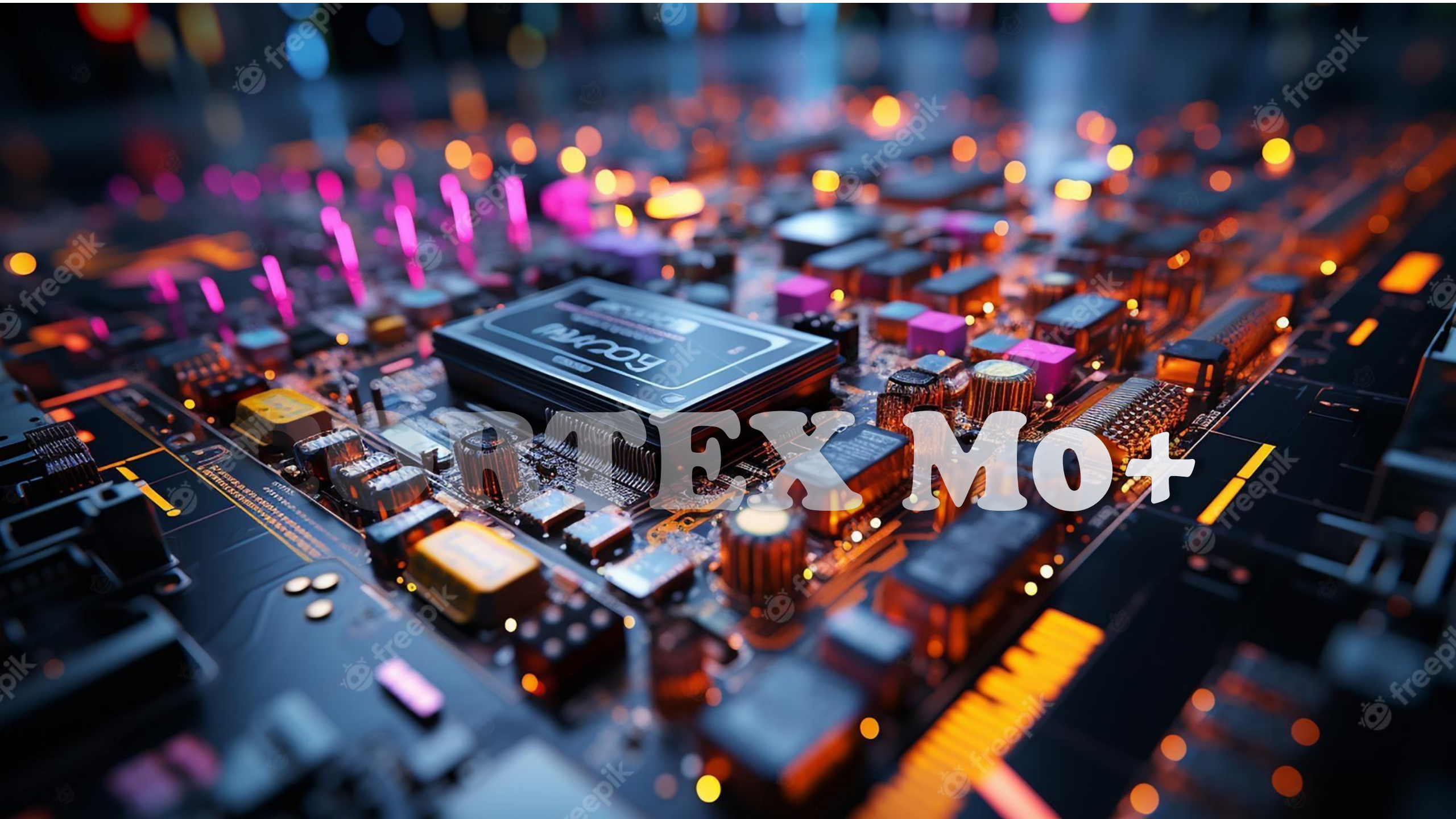
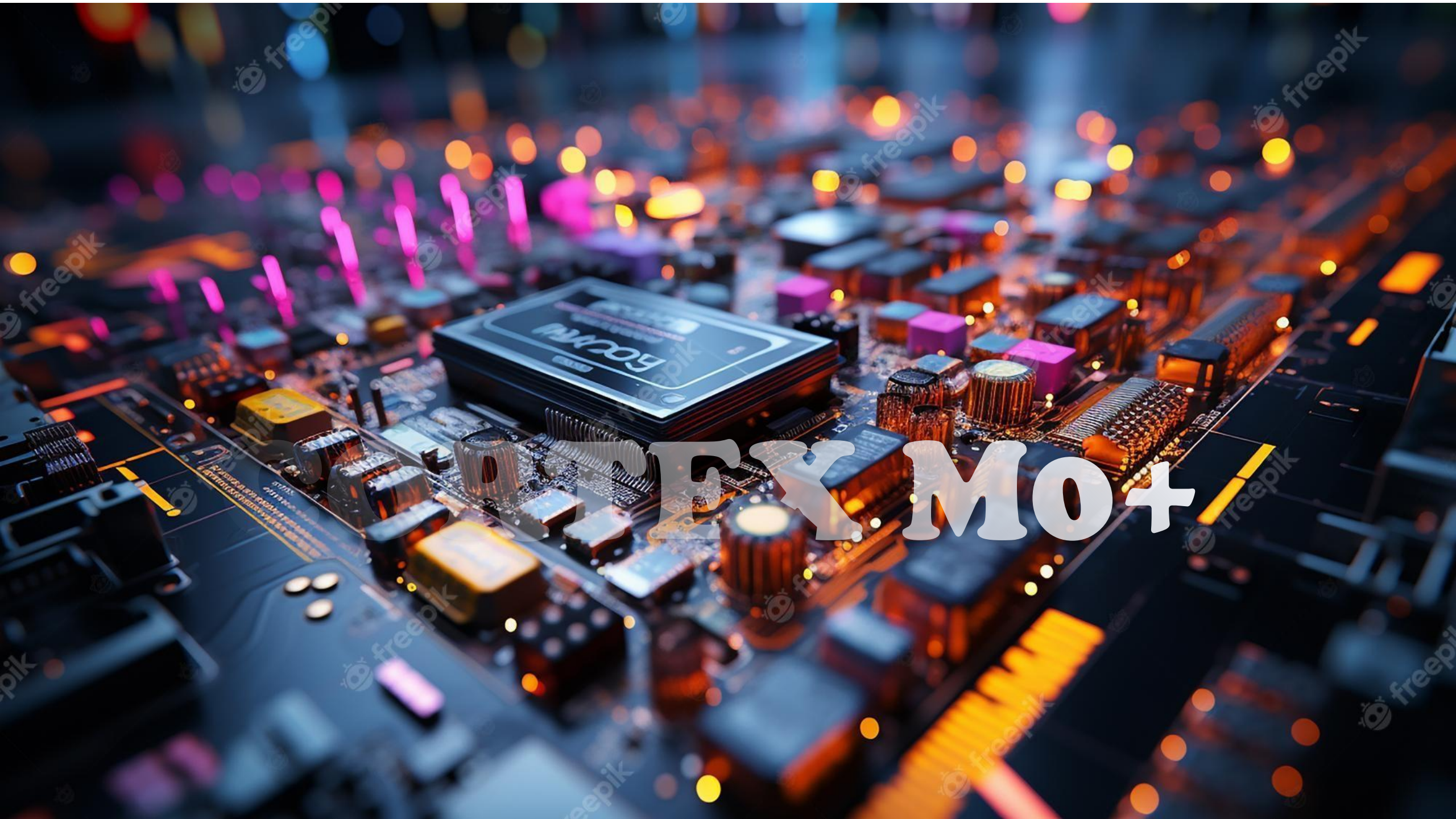


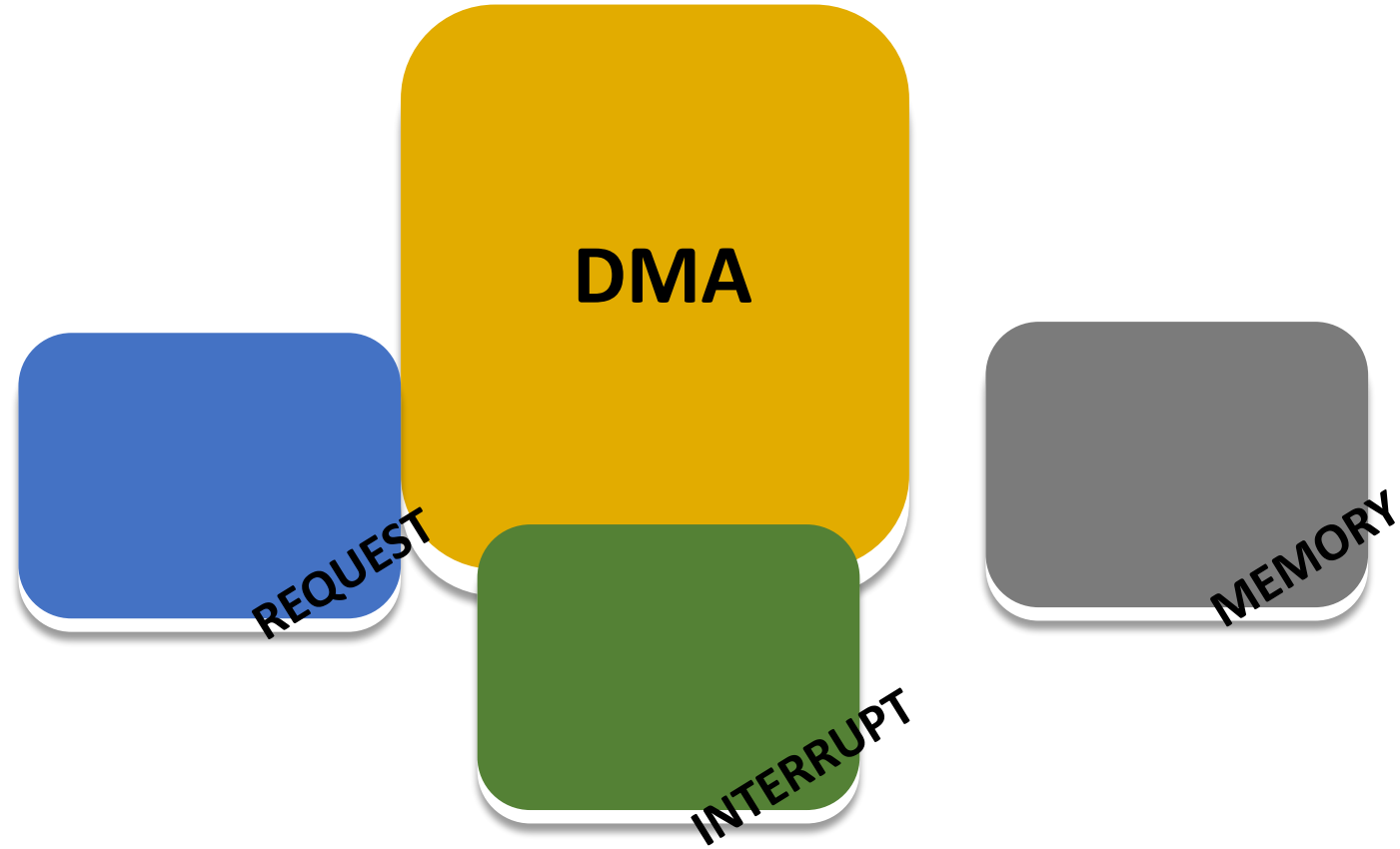
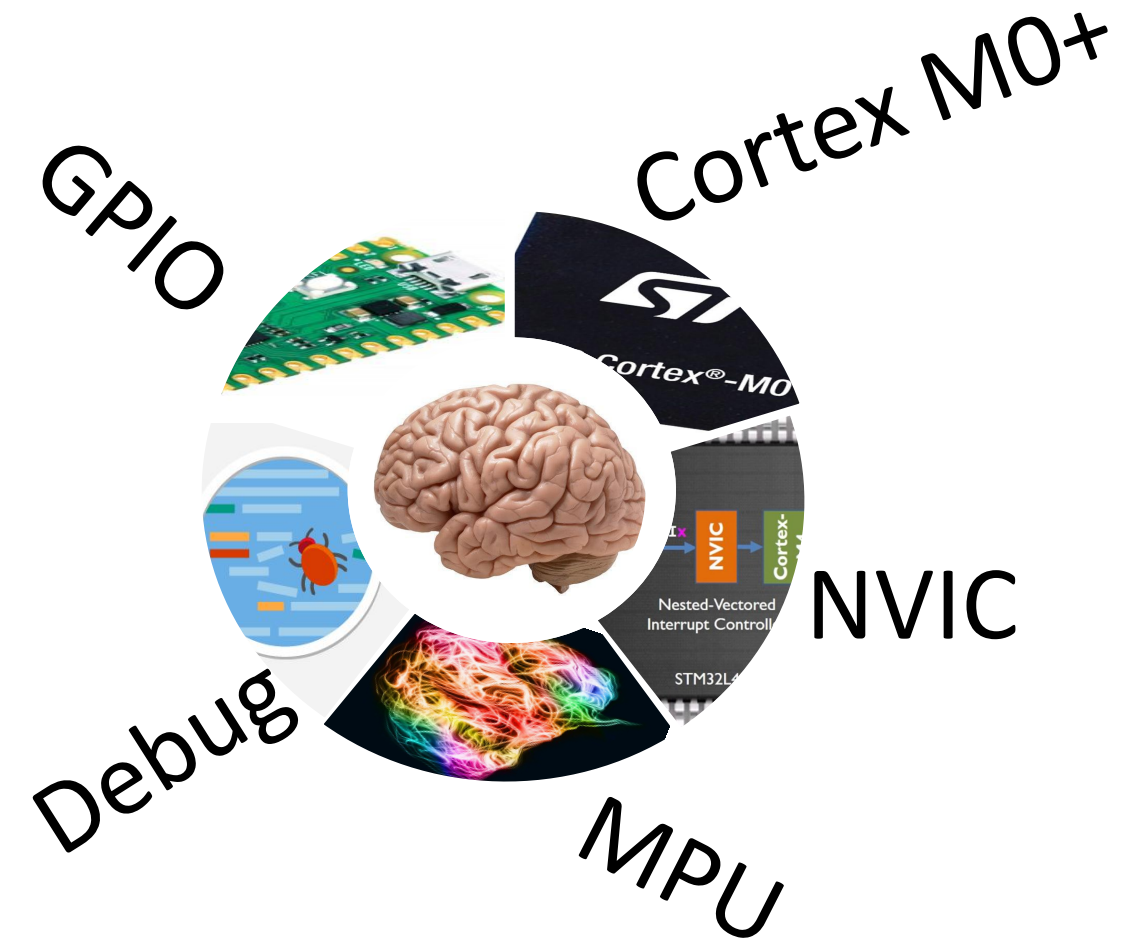


RTX Mo+



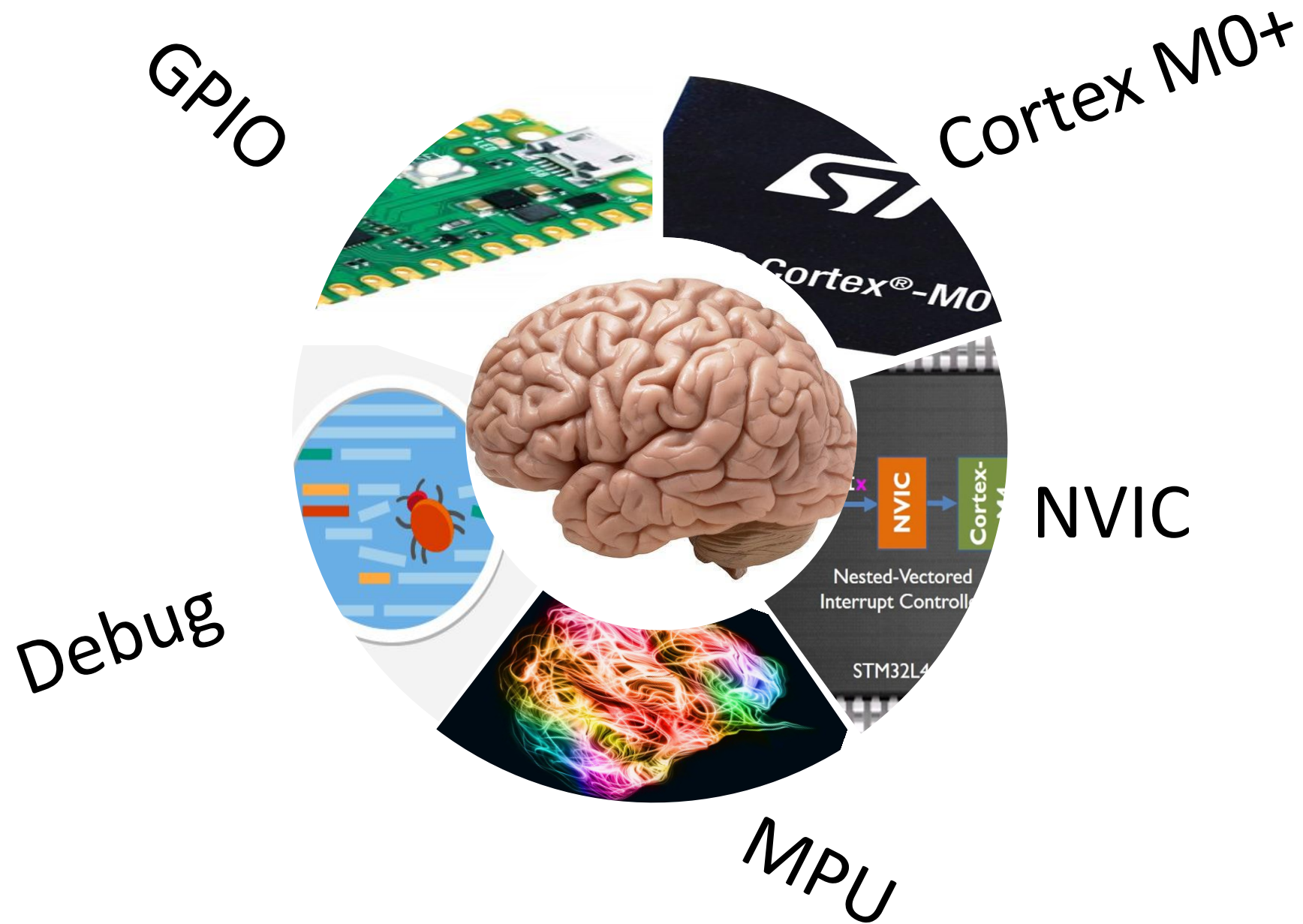


FREE MO+



**CORTEX M0+ & DMA**

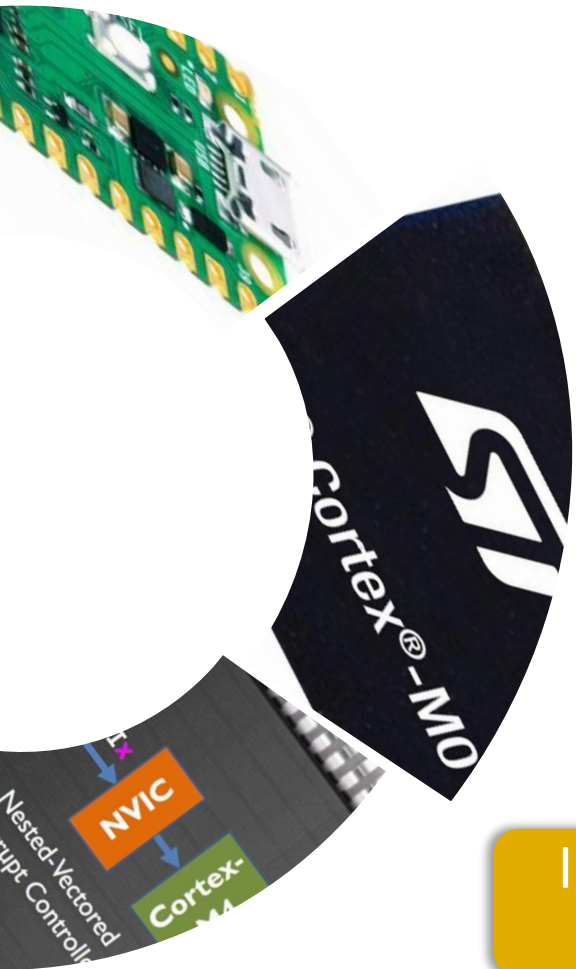






## Cortex M0+

The ARM Cortex-M0+ processor is a very low gate count, highly **energy efficient** processor that is intended for microcontroller and **deeply embedded** applications that require an area optimized, **low-power processor**.



# Cortex M0+ : Function

Intended Learning  
Outcome

Discuss the features of Cortex

# Features

- Tight integration
- Thumb instruction set (16 bit)
- Single-cycle I/O
- Power control optimization
- Integrated sleep modes
- Fast code execution
- Optimized code fetching
- Hardware multiplier
- Deterministic, high-performance interrupt
- Deterministic instruction cycle

# Interfaces

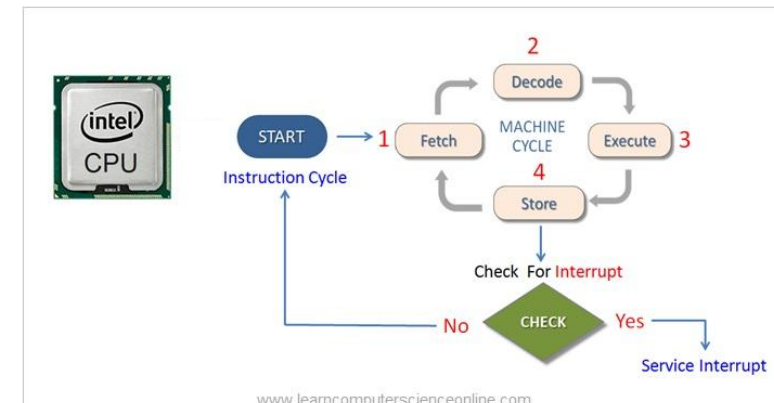
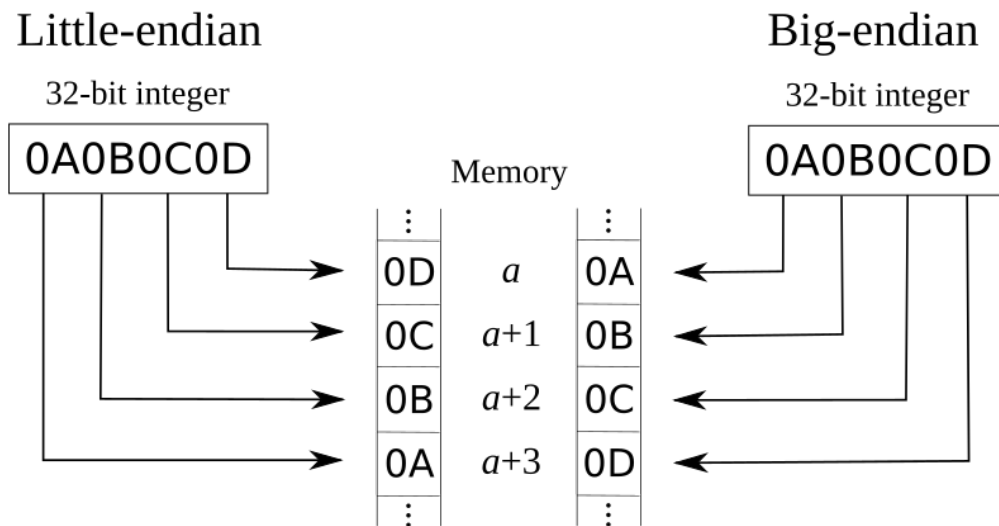
- External AHB-Lite interface to bus fabric
- Debug Access Port (DAP)
- Single-cycle I/O Port to SIO peripherals



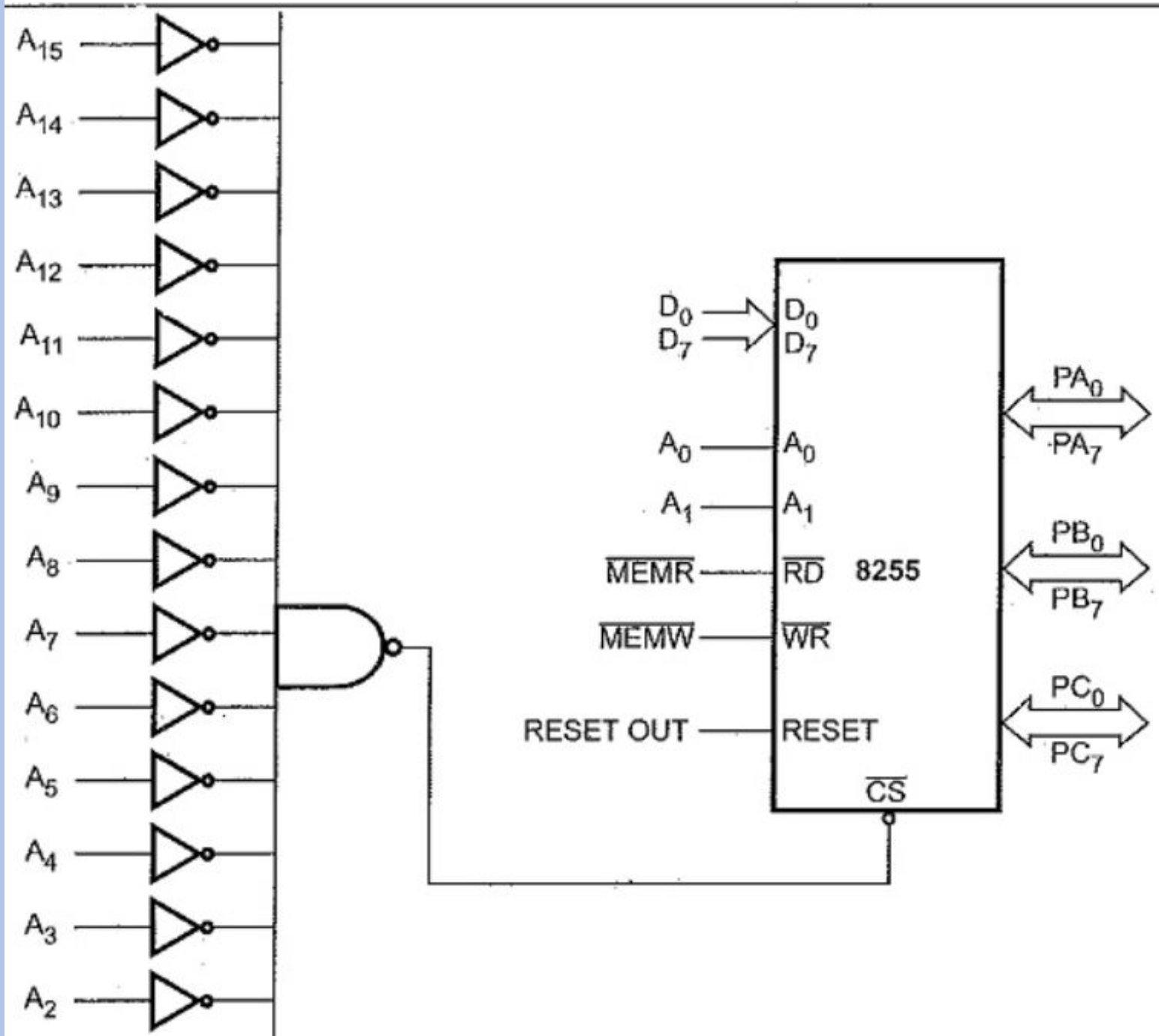
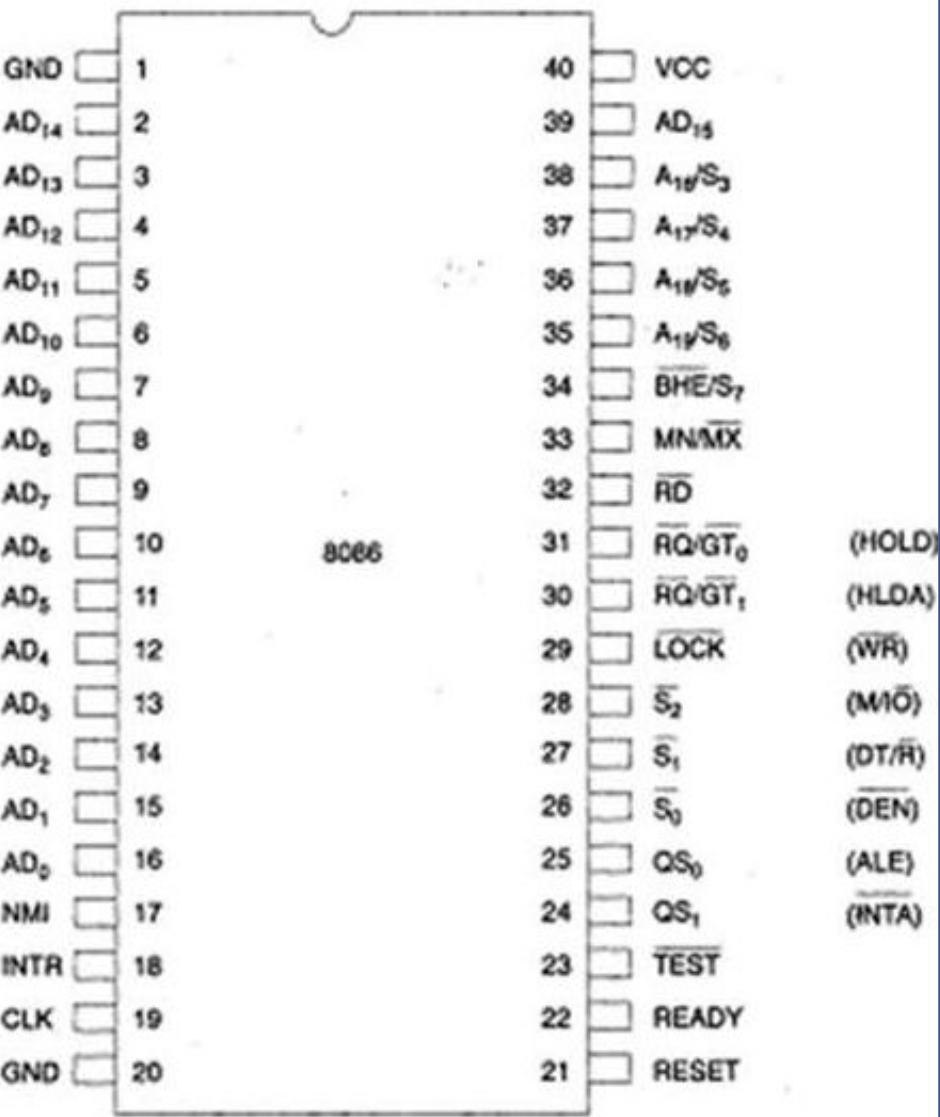
# Configuration

- **Architectural clock gating** (disable the parts of the chip when not in use - for power saving)
- **Little Endian bus access** – *Simplifying memory access (LSB to lowest address)*

## IOPORT:



- **Four Breakpoints**



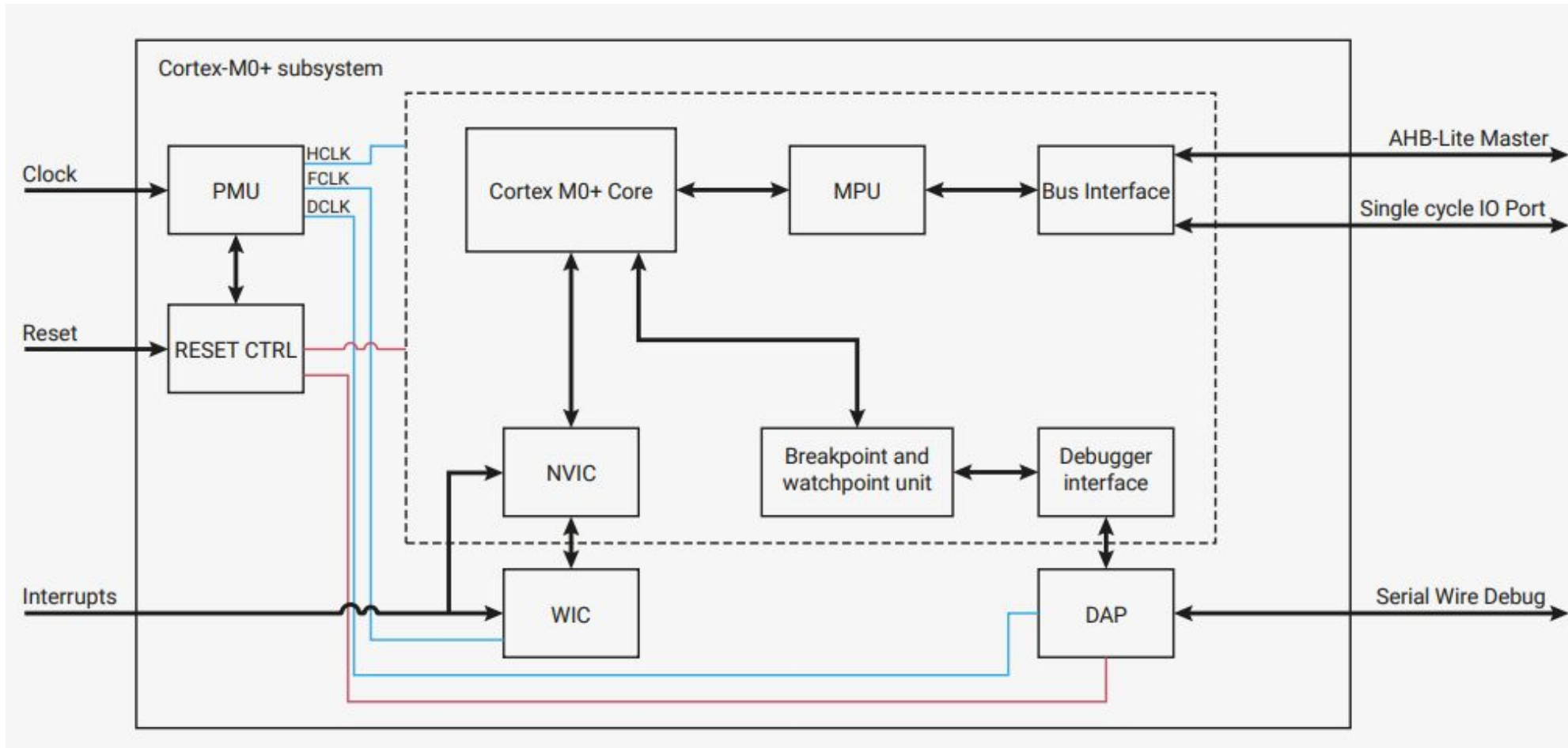
# Configuration

- **26 interrupts**
- **8 MPU regions**
  - (Memory Protection Unit) regions allow the microcontroller to protect different areas of memory from unauthorized access
- **All registers reset on powerup** (to know the status on boot up)
- **Fast multiplier** (hardware)
- **SysTick timer** (precise timekeeping mechanism)
- **Vector Table Offset Register** (This register specifies the location of the interrupt vector table)
- **34 WIC** (Wake-up Interrupt Controller)
- **DAP** (Debug Access Port, which provides a standardized interface for debugging and programming the microcontroller.)



# Functional Description

32-bit RISC processor



# Features

- The ARMv6-M Thumb<sup>®</sup> instruction set.
- Thumb-2 technology.
- An ARMv6-M compliant 24-bit SysTick timer.
- A 32-bit hardware multiplier. This is the standard single-cycle multiplier
- The ability to have deterministic, fixed-latency, interrupt handling.
- Load/store multiple instructions that can be abandoned and restarted to facilitate rapid interrupt handling.
- C Application Binary Interface compliant exception model. This is the ARMv6-M, C Application Binary Interface (CABI) compliant exception model that enables the use of pure C functions as interrupt handlers.
- Low power sleep-mode entry us

## NVIC features

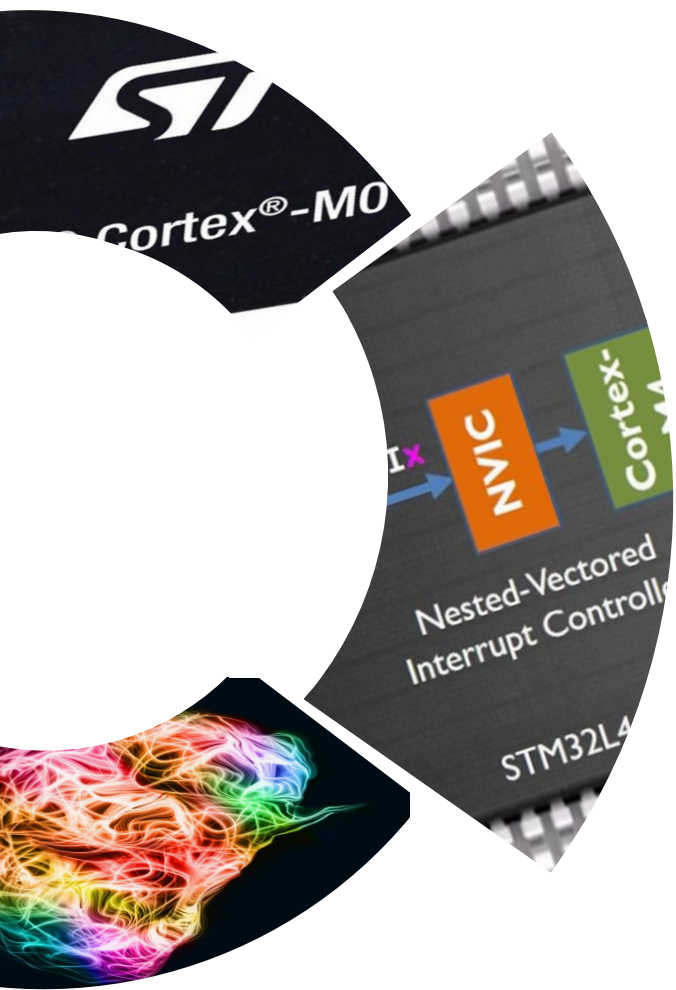
The Nested Vectored Interrupt Controller (NVIC) features are:

- 26 external interrupt inputs, each with four levels of priority.
- Dedicated Non-Maskable Interrupt (NMI) input (which can be driven from any standard interrupt source)
- Support for both level-sensitive and pulse-sensitive interrupt lines.
- Wake-up Interrupt Controller (WIC), providing ultra-low power sleep mode support.
- Relocatable vector table.

## SIO

1. An IR sensor is connected to the RP2040 in GPIO7, find the address and the value to be given at the address to enable the GPIO7 as the input pin.
2. 30 LEDs are connected across the GPIOs, find the address to turn on all the LED and write the value to be given





# NVIC

Intended Learning  
Outcome

How and what registers  
are used in NVIC

# NVIC

External interrupt signals connect to the Nested Vectored Interrupt Controller (NVIC), and the NVIC prioritizes the interrupts.

Software can set the priority of each interrupt. The NVIC and the Cortex-M0+ processor core are closely coupled, providing *low latency* interrupt processing and efficient processing of late arriving interrupts

*All NVIC registers are only accessible using word transfers. Any attempt to read or write a halfword or byte individually is unpredictable.*

*“**Nested**” refers to the fact that interrupts can themselves be interrupted, by higher-priority interrupts. “**Vectored**” refers to the hardware dispatching each interrupt to a distinct handler routine*

## SysTick timer

- A 24-bit system timer (SysTick).
- Additional configurable priority SysTick interrupt.

## Low power modes

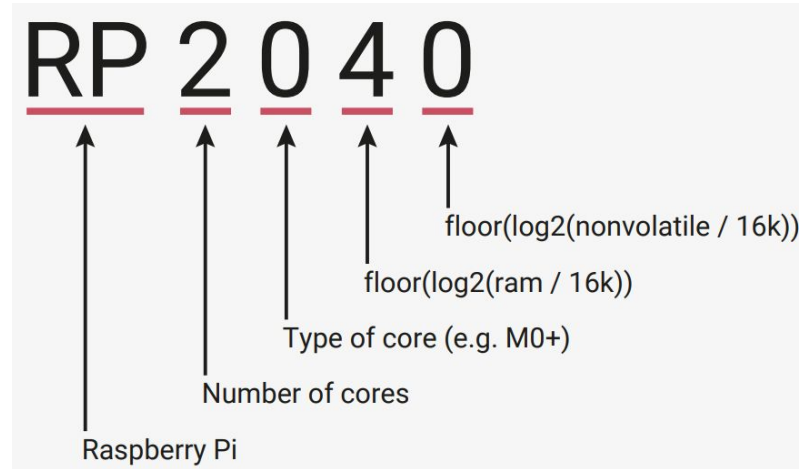
The implementation includes a WIC (Wakeup Interrupt Controller). This enables the processor and NVIC to be put into a very low-power sleep mode leaving the WIC to identify and prioritize interrupts.

The processor fully implements the Wait For Interrupt (WFI), Wait For Event (WFE) and the Send Event (SEV) instructions. In addition, the processor also supports the use of SLEEPONEXIT, that causes the processor core to enter sleep mode when it returns from an exception handler to Thread mode.

## NVIC register summary

| Name                  | Description                       |
|-----------------------|-----------------------------------|
| NVIC_ISER             | Interrupt Set-Enable Register.    |
| NVIC_ICER             | Interrupt Clear-Enable Register.  |
| NVIC_ISPR             | Interrupt Set-Pending Register.   |
| NVIC_ICPR             | Interrupt Clear-Pending Register. |
| NVIC_IPR0 - NVIC_IPR7 | Interrupt Priority Registers.     |

# Lets talk about memory



the "0" in the last position indicates that there is **no onboard non-volatile flash memory**, but it still includes the 16 KB ROM for essential functions.



**Total SRAM:** 264 KB

**Total ROM:** 16 KB.

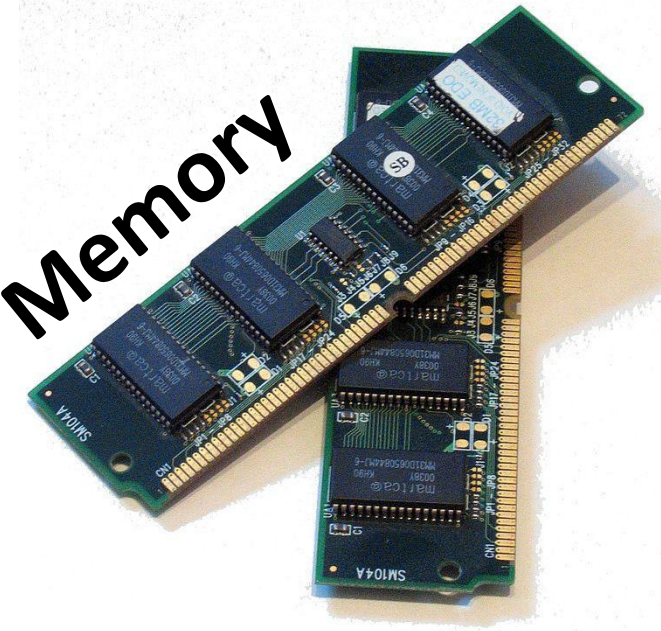
**XIP Flash Memory:** Typically up to 16 MB, though dependent on external flash.

**SRAM:** 264 KB

There are **four** 16k x 32-bit banks (64kB each) and **two** 1k x 32-bit banks (4kB each).

- **Striped** (interleaved) SRAM0-SRAM3 (higher performance due to faster memory access).
- **Non-striped** SRAM4-SRAM5 (sequential access).

**Memory**



# Address Map

|                               |            |
|-------------------------------|------------|
| ROM                           | 0x00000000 |
| XIP                           | 0x10000000 |
| SRAM                          | 0x20000000 |
| APB Peripherals               | 0x40000000 |
| AHB-Lite Peripherals          | 0x50000000 |
| IOPORT Registers              | 0xd0000000 |
| Cortex-M0+ internal registers | 0xe0000000 |



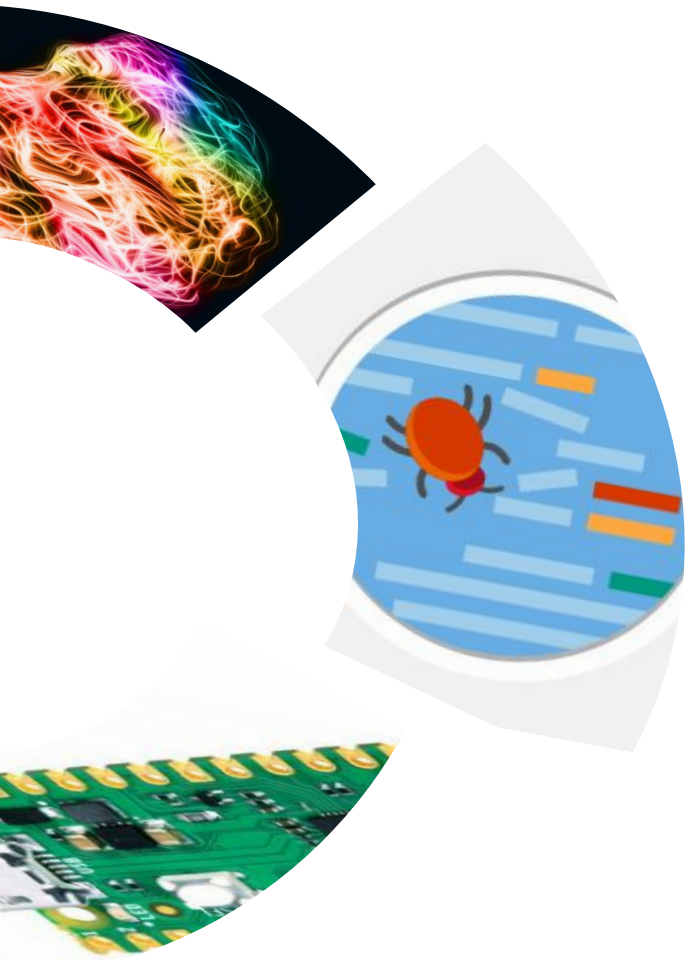
# MPU

Intended Learning  
Outcome

How and what registers  
are used in MPU

## MPU features

- Eight user-configurable memory regions.
- Eight sub-region disables per region.
- Execute never (XN) support.
- Default memory map support.



# DEBUG

Intended Learning  
Outcome

Debug How to set different debugging in RP2040 using their registers

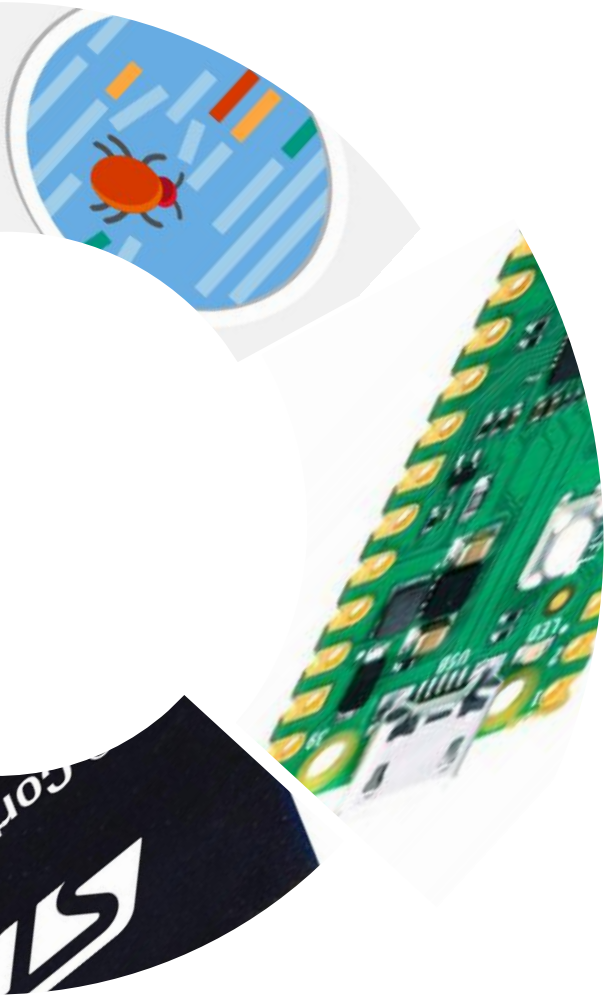


## Debug features

- Four hardware breakpoints.
- Two watchpoints.
- Program Counter Sampling Register (PCSR) for non-intrusive code profiling.
- Single step and vector catch capabilities.
- Support for unlimited software breakpoints using BKPT instruction.
- Non-intrusive access to core peripherals and zero-waitstate system slaves through a compact bus matrix. A debugger can access these devices, including memory, even when the processor is running.
- Full access to core registers when the processor is halted.
- Core Sight compliant debug access through a Debug Access Port (DAP) supporting Serial Wire debug connections.

## Debug Access Port

The processor is implemented with a low gate count Debug Access Port (DAP). The low gate count Debug Access Port (DAP) provides a Serial Wire debug-port, and connects to the processor slave port to provide full system-level debug access.



# GPIO

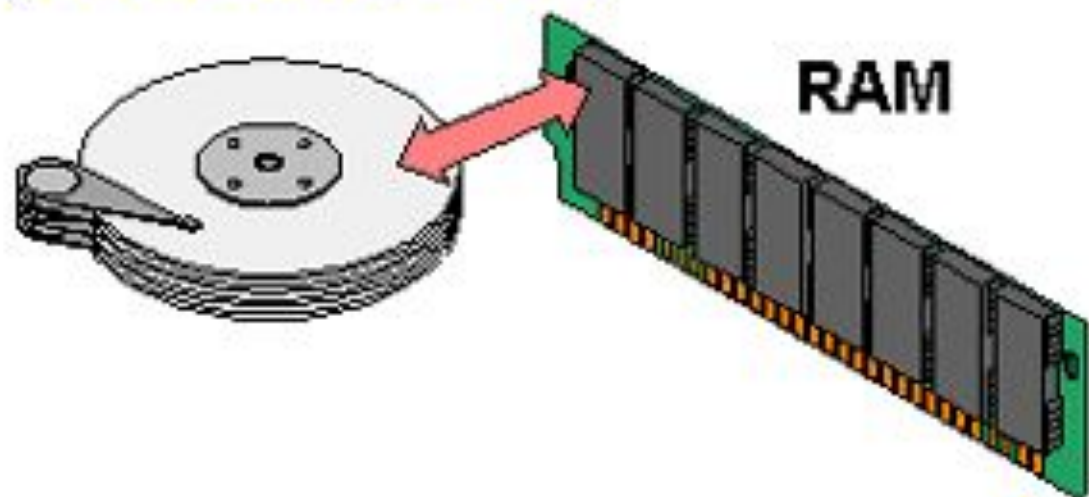
# DMA

**Direct Memory Access**

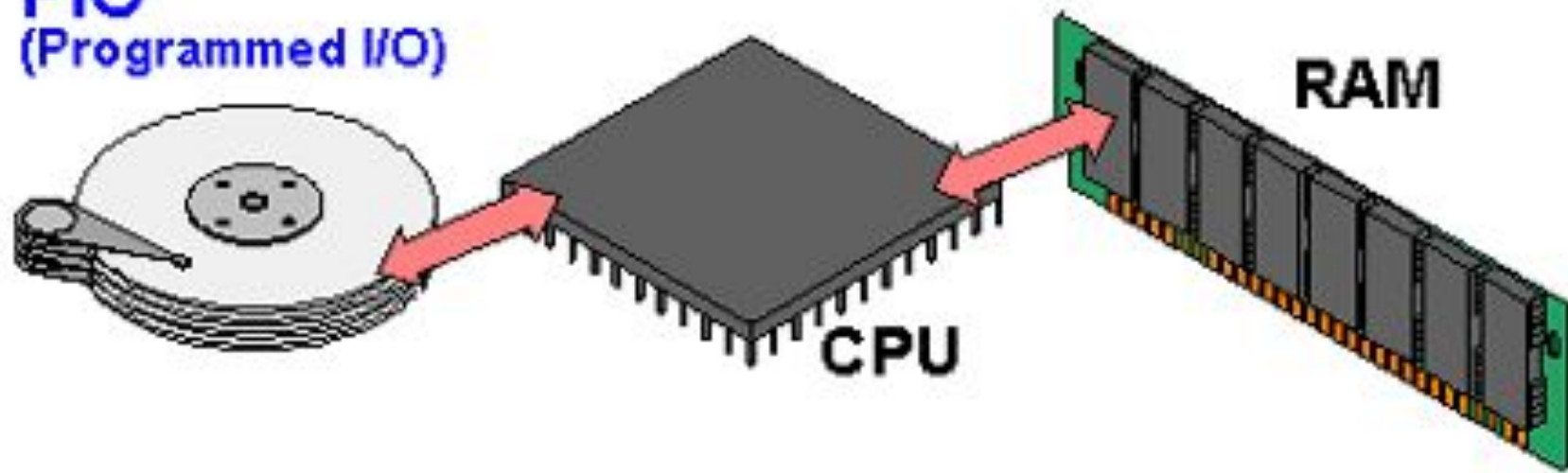
DMA:

- Data requests,
- Interrupts and
- Memory

## DMA (Direct Memory Access)



## PIO (Programmed I/O)



# Direct Memory Access

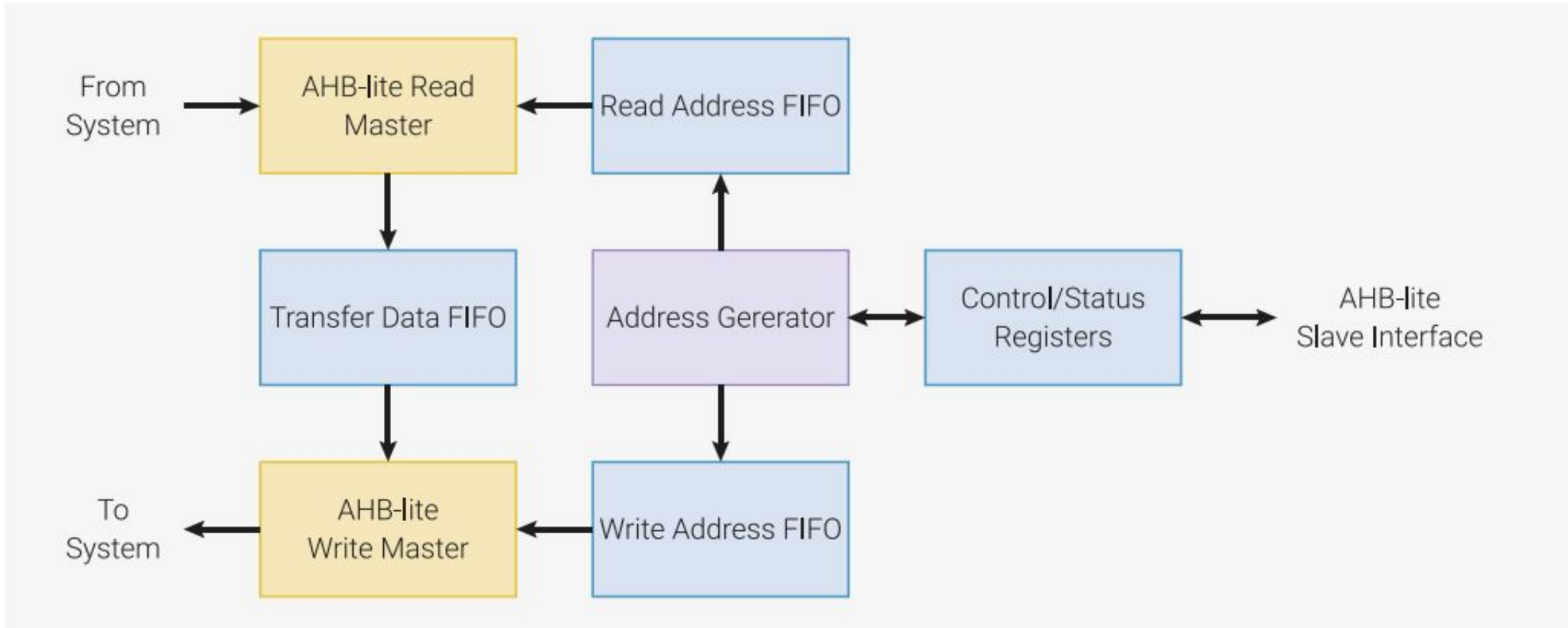
The RP2040 Direct Memory Access (DMA) controller has separate read and write master connections to the bus fabric, and performs bulk data transfers on a processor's behalf. This leaves processors free to attend to other tasks, or enter low-power sleep states.

Intended Learning  
Outcome

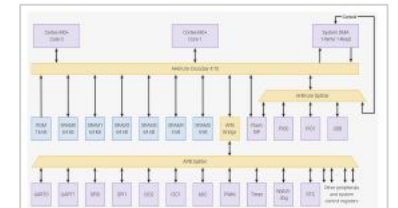
Illustrate Direct Memory Access



## Direct Memory Access

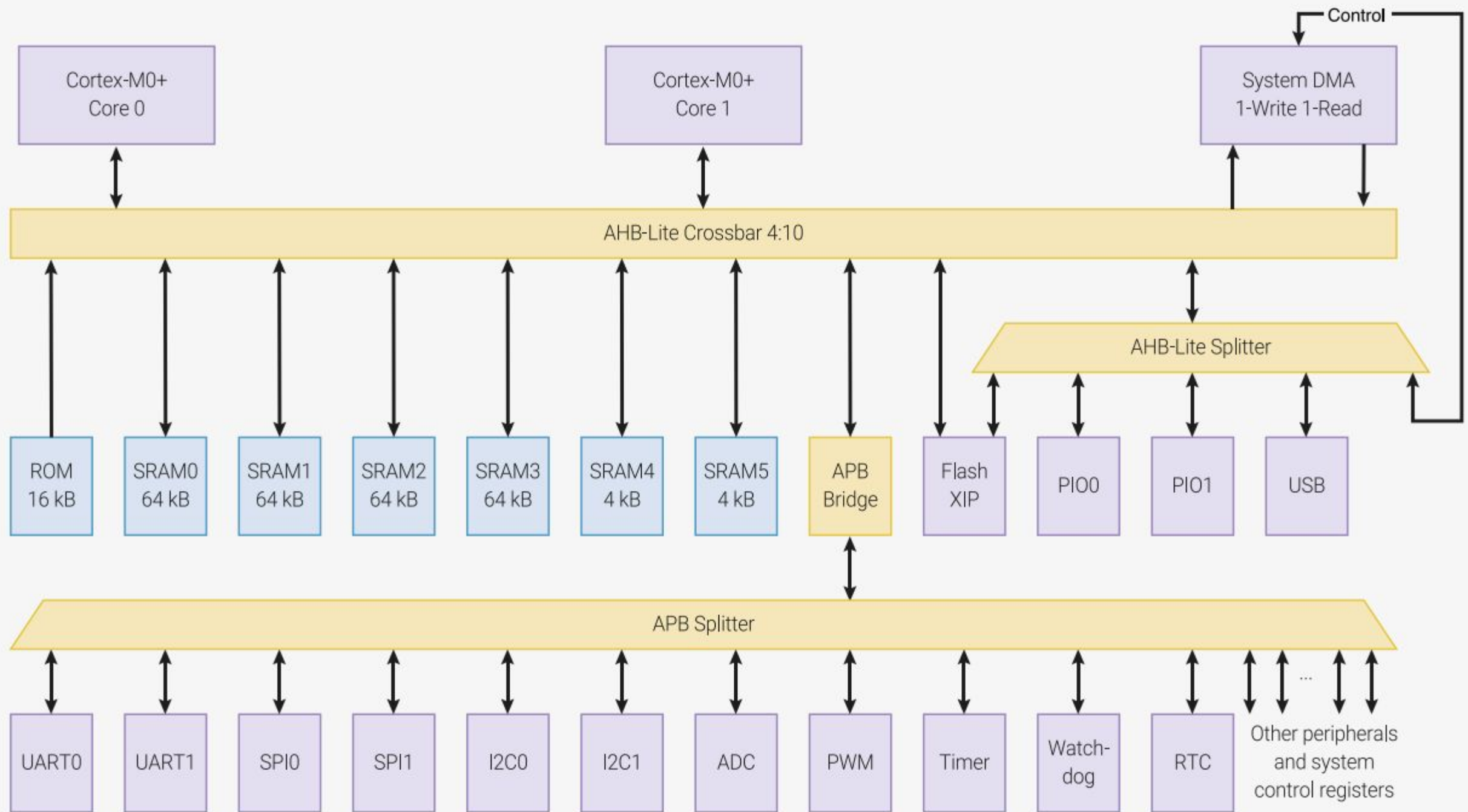


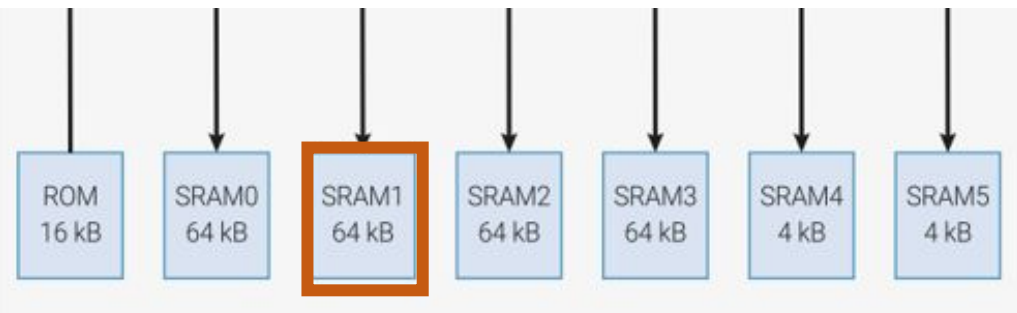
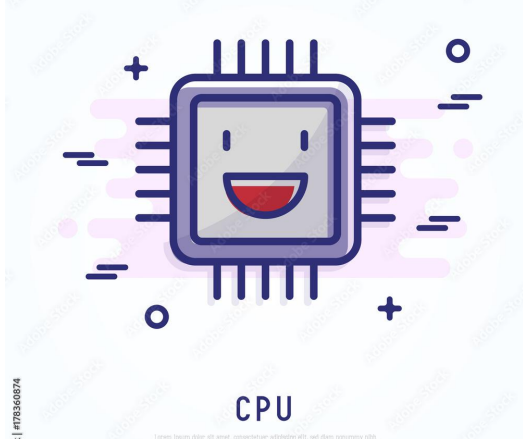
**32** bits in size, **one** read access and **one** write access every clock cycle

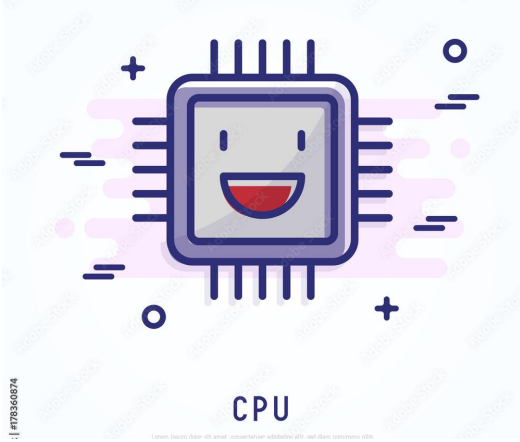


**There are 12 independent channels, each which supervise a sequence of bus transfers, usually in one of the following scenarios:**

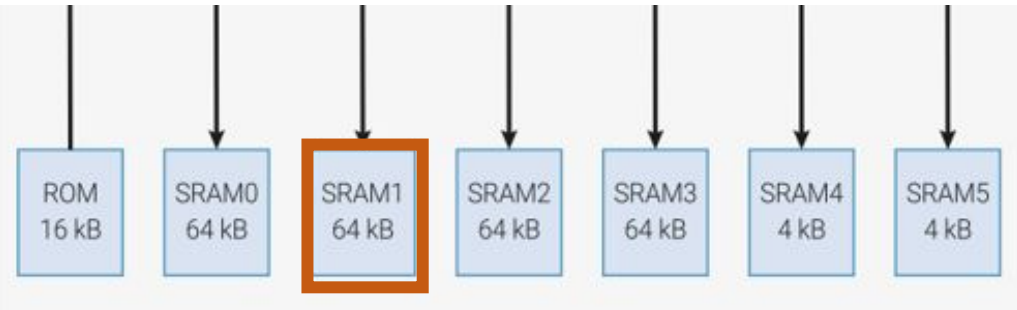
- **Memory-to-peripheral:** a peripheral signals the DMA when it needs more data to transmit. The DMA reads data from an array in RAM or flash, and writes to the peripheral's data FIFO.
- **Peripheral-to-memory:** a peripheral signals the DMA when it has received data. The DMA reads this data from the peripheral's data FIFO, and writes it to an array in RAM.
- **Memory-to-memory:** the DMA transfers data between two buffers in RAM, as fast as possible







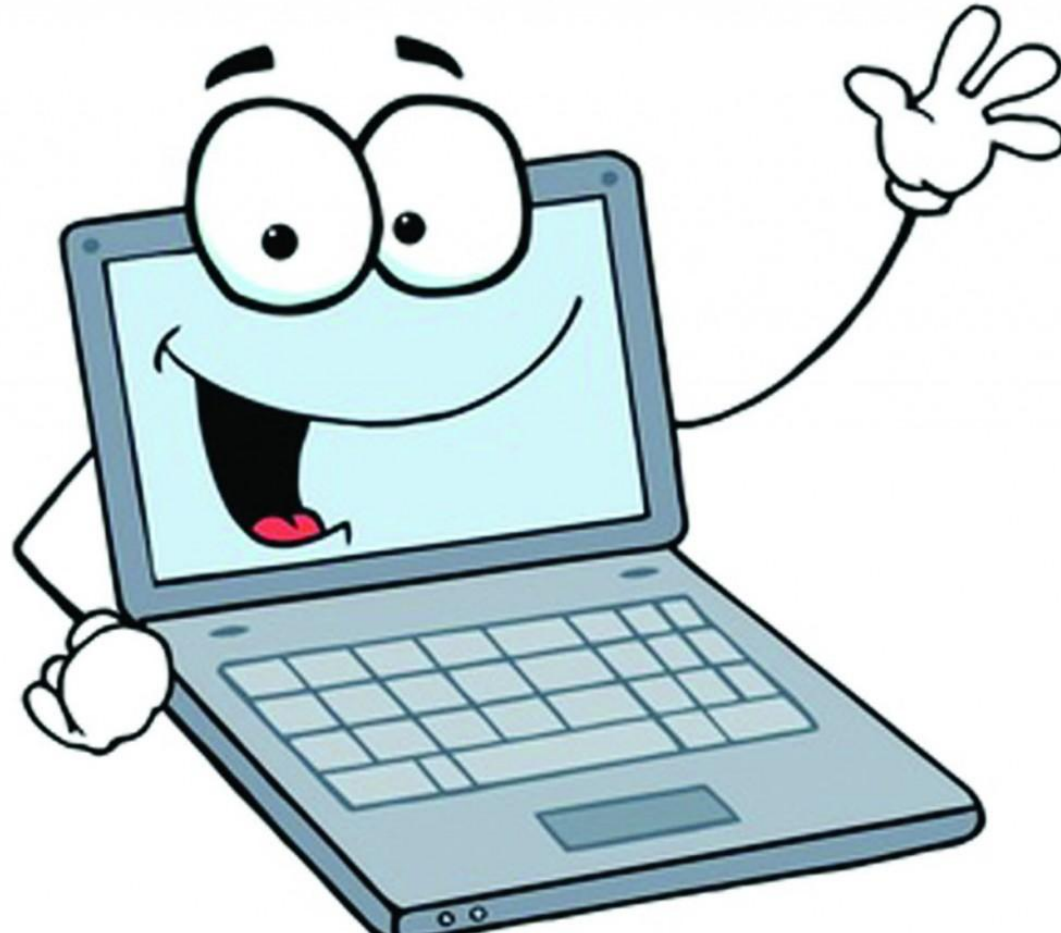
KidsToday.in



# Direct Memory Access

CONTROL AND STATUS  
REGISTERS

CHAIN\_TO



TRANSFER SIZE CAN BE  
EITHER 32, 16, OR 8 BITS

**We do all  
these to :**

Making the DMA more autonomous means that much less processor supervision is required: overall this allows the system to do more at once, or to dissipate less power.



# Configuring Channels

Each channel has four control/status registers

- **READ\_ADDR** is a pointer to the next address to be read
- **WRITE\_ADDR** is a pointer to the next address to be written to
- **TRANS\_COUNT** shows the number of transfers remaining in the current transfer sequence,
- **CTRL** is used to configure all other aspects of the channel's behaviour, to enable/disable it, and to check for completion.

# Configuring Channels

## Read and Write Addresses

- `READ_ADDR` and `WRITE_ADDR` contain the address the channel will next read from, and write to, respectively. (update automatically, increment by 1, 2 or 4 bytes at a time)
- Software should generally program these registers with new start addresses each time a new transfer sequence starts. If `READ_ADDR` and `WRITE_ADDR` are not reprogrammed, the DMA will use the current values as start addresses for the next transfer.

For example:

- If the address does not increment (e.g. it is the address of a peripheral FIFO), and the next transfer sequence is to/from that same address, there is no need to write to the register again.
- When transferring to/from a consecutive series of buffers in memory (e.g. scattering and gathering), an address register will already have incremented to the start of the next buffer at the completion of a transfer.

# Configuring Channels

## Transfer Count

- Reading from TRANS\_COUNT yields the number of transfers remaining in the current transfer sequence
- Each time the channel starts a new transfer sequence, the most recent value written to TRANS\_COUNT
- For debugging purposes, the last value written can be read from the DBG\_TCR
- If the channel is triggered multiple times without intervening writes to TRANS\_COUNT, it performs the same number of transfers each time.

# Configuring Channels

## Control/Status

- Configure the size - `CTRL.DATA_SIZE`
- Configure if and how read and write - `CTRL.INCR_WRITE`, `CTRL.INCR_READ`, `CTRL.RING_SEL`, `CTRL.RING_SIZE`
- Another channel (or none) to be triggered - `CTRL.CHAIN_TO`
- Peripheral data request (DREQ) signal - `CTRL.TREQ_SEL`
- See when the channel is idle - `CTRL.BUSY`
- Channel has encountered a bus error - `CTRL.AHB_ERROR`, `CTRL.READ_ERROR`, or `CTRL.WRITE_ERROR`

# Starting Channels

## 3 ways to start a channel

- Writing to a channel trigger register
- A chain trigger from another channel which has just completed, and has its CHAIN\_TO field configured
- The MULTI\_CHAN\_TRIGGER register, which can start multiple channels at once

## Aliases and Triggers

Table 118. Control register aliases. Each channel has four control/status registers. Each register can be accessed at multiple different addresses. In each naturally-aligned group of four, all four registers appear, in different orders.

| Offset         | +0x0      | +0x4        | +0x8        | +0xC (Trigger)   |
|----------------|-----------|-------------|-------------|------------------|
| 0x00 (Alias 0) | READ_ADDR | WRITE_ADDR  | TRANS_COUNT | CTRL_TRIG        |
| 0x10 (Alias 1) | CTRL      | READ_ADDR   | WRITE_ADDR  | TRANS_COUNT_TRIG |
| 0x20 (Alias 2) | CTRL      | TRANS_COUNT | READ_ADDR   | WRITE_ADDR_TRIG  |
| 0x30 (Alias 3) | CTRL      | WRITE_ADDR  | TRANS_COUNT | READ_ADD_TRIG    |

# Starting Channels

## Trigger registers do not start the channel if

- The channel is disabled via CTRL.EN
- The channel is already running
- The value 0 is written to the trigger register



# Starting Channels

## Chaining

- When a channel completes, it can name a different channel to immediately be triggered. This can be used as a callback for the second channel to reconfigure and restart the first.

**This feature is configured through the CHAIN\_TO field in the channel CTRL register.**

**"ping-pong" configuration**

# Starting Channels

## Null Triggers and Chain Interrupts

- writing all-zeroes to a trigger register does not start the channel. This is called a null trigger, and it has two purposes:
  - Cause a halt at the end of an array of control blocks, by appending an all-zeroes block
  - Reduce the number of interrupts generated when control blocks are used

By default, a channel will generate an interrupt each time it finishes a transfer sequence, unless that channel's **IRQ** is masked in **INTE0 or INTE1**

The channel **CTRL** register has a field called **IRQ\_QUIET**

# Data Request (DREQ)

Peripherals produce or consume data at their own pace. If the DMA simply transferred data as fast as possible, loss or corruption of data would ensue. DREQs are a communication channel between peripherals and the DMA, which enables the DMA to pace transfers according to the needs of the peripheral.

# Data Request (DREQ)

Use of DREQ in DMA

Intended Learning Outcome

| DREQ | DREQ Channel  | DREQ | DREQ Channel  | DREQ | DREQ Channel   | DREQ | DREQ Channel    |
|------|---------------|------|---------------|------|----------------|------|-----------------|
| 0    | DREQ_PI00_TX0 | 10   | DREQ_PI01_TX2 | 20   | DREQ_UART0_TX  | 30   | DREQ_PWM_WRAP6  |
| 1    | DREQ_PI00_TX1 | 11   | DREQ_PI01_TX3 | 21   | DREQ_UART0_RX  | 31   | DREQ_PWM_WRAP7  |
| 2    | DREQ_PI00_TX2 | 12   | DREQ_PI01_RX0 | 22   | DREQ_UART1_TX  | 32   | DREQ_I2C0_TX    |
| 3    | DREQ_PI00_TX3 | 13   | DREQ_PI01_RX1 | 23   | DREQ_UART1_RX  | 33   | DREQ_I2C0_RX    |
| 4    | DREQ_PI00_RX0 | 14   | DREQ_PI01_RX2 | 24   | DREQ_PWM_WRAP0 | 34   | DREQ_I2C1_TX    |
| 5    | DREQ_PI00_RX1 | 15   | DREQ_PI01_RX3 | 25   | DREQ_PWM_WRAP1 | 35   | DREQ_I2C1_RX    |
| 6    | DREQ_PI00_RX2 | 16   | DREQ_SPI0_TX  | 26   | DREQ_PWM_WRAP2 | 36   | DREQ_ADC        |
| 7    | DREQ_PI00_RX3 | 17   | DREQ_SPI0_RX  | 27   | DREQ_PWM_WRAP3 | 37   | DREQ_XIP_STREAM |
| 8    | DREQ_PI01_TX0 | 18   | DREQ_SPI1_TX  | 28   | DREQ_PWM_WRAP4 | 38   | DREQ_XIP_SSITX  |
| 9    | DREQ_PI01_TX1 | 19   | DREQ_SPI1_RX  | 29   | DREQ_PWM_WRAP5 | 39   | DREQ_XIP_SSIRX  |

# Credit-based DREQ Scheme

**The RP2040 DMA is designed for systems where:**

*The area and power cost of large peripheral data FIFOs is prohibitive*

*The bandwidth demands of individual peripherals may be high, e.g. >50% bus injection rate for short periods*

*Bus latency is low, but multiple masters may be competing for bus access*

# Direct Memory Access

## Addressing 'em

The DMA registers start at a base address of `0x50000000` (defined as `DMA_BASE` in SDK).

| Offset | Name                         | Info                                |
|--------|------------------------------|-------------------------------------|
| 0x000  | <code>CH0_READ_ADDR</code>   | DMA Channel 0 Read Address pointer  |
| 0x004  | <code>CH0_WRITE_ADDR</code>  | DMA Channel 0 Write Address pointer |
| 0x008  | <code>CH0_TRANS_COUNT</code> | DMA Channel 0 Transfer Count        |
| 0x00c  | <code>CH0_CTRL_TRIG</code>   | DMA Channel 0 Control and Status    |
| 0x010  | <code>CH0_AL1_CTRL</code>    | Alias for channel 0 CTRL register   |

# Direct Memory Access

## Configure 'em

**DMA:** CH0\_CTRL\_TRIG, CH1\_CTRL\_TRIG, ..., CH10\_CTRL\_TRIG, CH11\_CTRL\_TRIG Registers

**Offsets:** 0x00c, 0x04c, ..., 0x28c, 0x2cc

**Description**

DMA Channel N Control and Status

| Bits | Name      | Description                                                                                                                                     | Type | Reset |
|------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 31   | AHB_ERROR | Logical OR of the READ_ERROR and WRITE_ERROR flags. The channel halts when it encounters any bus error, and always raises its channel IRQ flag. | RO   | 0x0   |



| Bits  | Name        | Description                                                                                                                                                                                                                                                                                              | Type | Reset |
|-------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 30    | READ_ERROR  | If 1, the channel received a read bus error. Write one to clear.<br>READ_ADDR shows the approximate address where the bus error was encountered (will not to be earlier, or more than 3 transfers later)                                                                                                 | WC   | 0x0   |
| 29    | WRITE_ERROR | If 1, the channel received a write bus error. Write one to clear.<br>WRITE_ADDR shows the approximate address where the bus error was encountered (will not to be earlier, or more than 5 transfers later)                                                                                               | WC   | 0x0   |
| 28:25 | Reserved.   | -                                                                                                                                                                                                                                                                                                        | -    | -     |
| 24    | BUSY        | This flag goes high when the channel starts a new transfer sequence, and low when the last transfer of that sequence completes. Clearing EN while BUSY is high pauses the channel, and BUSY will stay high while paused.<br><br>To terminate a sequence early (and clear the BUSY flag), see CHAN_ABORT. | RO   | 0x0   |
| 23    | SNIFF_EN    | If 1, this channel's data transfers are visible to the sniff hardware, and each transfer will advance the state of the checksum. This only applies if the sniff hardware is enabled, and has this channel selected.<br><br>This allows checksum to be enabled or disabled on a per-                      | RW   | 0x0   |

|       |           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |    |      |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|------|
| 22    | BSWAP     | <p>Apply byte-swap transformation to DMA data.</p> <p>For byte data, this has no effect. For halfword data, the two bytes of each halfword are swapped. For word data, the four bytes of each word are swapped to reverse order.</p>                                                                                                                                                                                                                                                                         | RW | 0x0  |
| 21    | IRQ_QUIET | <p>In QUIET mode, the channel does not generate IRQs at the end of every transfer block. Instead, an IRQ is raised when NULL is written to a trigger register, indicating the end of a control block chain.</p> <p>This reduces the number of interrupts to be serviced by the CPU when transferring a DMA chain of many small control blocks.</p>                                                                                                                                                           | RW | 0x0  |
| 20:15 | TREQ_SEL  | <p>Select a Transfer Request signal.</p> <p>The channel uses the transfer request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system).</p> <p>0x0 to 0x3a → select DREQ n as TREQ</p> <p>0x3b → Select Timer 0 as TREQ</p> <p>0x3c → Select Timer 1 as TREQ</p> <p>0x3d → Select Timer 2 as TREQ (Optional)</p> <p>0x3e → Select Timer 3 as TREQ (Optional)</p> <p>0x3f → Permanent request, for unpaced transfers.</p> | RW | 0x00 |

| Bits  | Name       | Description                                                                                                                                                                                                                                                                                                                                                                                      | Type | Reset |
|-------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| 14:11 | CHAIN_TO   | When this channel completes, it will trigger the channel indicated by CHAIN_TO. Disable by setting CHAIN_TO = <i>(this channel)</i> .                                                                                                                                                                                                                                                            | RW   | 0x0   |
| 10    | RING_SEL   | Select whether RING_SIZE applies to read or write addresses.<br>If 0, read addresses are wrapped on a $(1 \ll \text{RING\_SIZE})$ boundary. If 1, write addresses are wrapped.                                                                                                                                                                                                                   | RW   | 0x0   |
| 9:6   | RING_SIZE  | Size of address wrap region. If 0, don't wrap. For values $n > 0$ , only the lower $n$ bits of the address will change. This wraps the address on a $(1 \ll n)$ byte boundary, facilitating access to naturally-aligned ring buffers.<br><br>Ring sizes between 2 and 32768 bytes are possible. This can apply to either read or write addresses, based on value of RING_SEL.<br>0x0 → RING_NONE | RW   | 0x0   |
| 5     | INCR_WRITE | If 1, the write address increments with each transfer. If 0, each write is directed to the same, initial address.<br><br>Generally this should be disabled for memory-to-peripheral transfers.                                                                                                                                                                                                   | RW   | 0x0   |
| 4     | INCR_READ  | If 1, the read address increments with each transfer. If 0, each read is directed to the same, initial address.                                                                                                                                                                                                                                                                                  | RW   | 0x0   |

# Sniff Hardware

Sniff hardware is a generic term used to describe a piece of hardware or a component in a computer system that is capable ***of observing, monitoring, or interacting with data transfers on specific channels*** or buses.

A checksum is a mathematical value calculated from the data to check for errors or verify data integrity. The statement suggests that the sniff hardware may be involved in the calculation or monitoring of this checksum as data is transferred

**In A DMA device, what value should I write to the control and status register and at which address,  
If:**

1. The DMA device is connected to channel 2
2. There is a write error
3. The bus is busy
4. Sniff is enabled
5. Half word swap is demanded
6. Interrupt is set to be in quiet mode
7. Transfer request is from UART0\_Tx

**In A DMA device, what value should I write to the control and status register and at which address,  
If:**

8. Set chain to be in ping pong configuration with channel 4
9. No ring configuration
10. The operation is between a i2c and memory device.
11. 16bit data is transferred
12. I2c device connected must be given the top priority

**In A DMA device, what value should I write to the control and status register and at which address,  
If:**

1. The DMA device is connected to channel 4
2. There is a read error
3. The bus is not busy
4. Sniff is enabled
5. Half word swap is demanded
6. Interrupt is set to be in quiet mode
7. Transfer request is from UART0\_Tx



**In A DMA device, what value should I write to the control and status register and at which address,  
If:**

8. Set chain to be in ping pong configuration with channel 1
9. No ring configuration
10. The operation is between a i2c and memory device.
11. 16bit data is transferred
12. I2c device connected must be given the priority next to the power interrupt

The DMA registers start at a base address of **0x50000000**

**CH2\_CTRL\_TRIG**

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 1  | 0  | 1  | x  | x  | x  | x  | 1  | 1  | 0  | 0  | 1  | 1  | 1  | 1  | 1  | 0  | 0  | 1  | 0  | 0  | x  | x | x | x | x | 1 | 0 | 1 | 0 | 1 | 1 |
| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |

Register name: CH1\_CTRL\_TRIG

Base Address: 0x50000000

Offset Address: